

All You Need Is Addition

A Logarithmic Number System Acceleration Framework for CKKS

Dimitrios Schoinianakis 

Nokia Bell Labs, Athens, Greece

Abstract

A practical acceleration framework for CKKS homomorphic encryption is proposed, in which multiplication-heavy sub-circuits are evaluated by adding encrypted log-magnitudes rather than multiplying ciphertexts. This logarithmic-number-system (LNS) representation consumes no multiplicative levels; a lightweight interactive refresh operation re-enters the linear CKKS domain whenever additive accumulation is required, avoiding bootstrapping entirely. Three execution strategies—an automatic planner (`lns-auto`), a client-side accumulation variant (`lns-dda`), and a server-side re-encryption variant (`lns-ddr`)—are mechanized in an `OpenFHE`-based runtime under `HEStd_128_classic` security and evaluated on a deep multiplication chain and an attention-like pipeline across three network environments. On the attention pipeline, the linear baseline requires a ring degree $N = 65,536$ and approximately 2.9 GB of public-context material, whereas LNS operates at $N = 8,192$ with only about 50 MB—a $58\times$ context reduction yielding $22\text{--}36\times$ end-to-end speedup and up to $46\times$ payload reduction. On the multiplication chain the gap widens with depth, from $\sim 10\times$ speedup and $26\times$ payload reduction at $L=8$ matrices to $35\times$ and $98\times$ at $L=20$, because LNS context requirements are decoupled from multiplicative depth while the linear baseline must escalate its ring degree to maintain accuracy and security.

Keywords: homomorphic encryption, CKKS, logarithmic number system, secure inference

1 Introduction

CKKS is nowadays the dominant approximate homomorphic encryption scheme for vectorized arithmetic over real and complex data [CKKS17]. Although it supports packed fixed-point operations elegantly, its efficiency remains constrained by ciphertext-ciphertext multiplication: each such operation triggers polynomial multiplication in the ring, re-linearization via key switching, and rescaling to manage scale growth. In practice, the asymmetry between addition and multiplication is large enough that many HE system designs are organized around minimizing multiplicative depth, amortizing key-switching overhead, or inserting bootstrapping phases only when unavoidable [CHK⁺18]. This work investigates a complementary direction: instead of optimizing CKKS multiplication itself, the representation is changed to a logarithmic number system (LNS), in which multiplication becomes addition on log-magnitudes, while sign handling is separated as metadata [Par20]. When combined with CKKS, this permits multiplicative subcircuits to be evaluated in the log domain and refreshed back to ordinary linear CKKS only when additive accumulation is required.

The resulting protocol family is interactive: the server sends encrypted log-domain intermediates to the client and the client decrypts and exponentiates them to retrieve their

Email: dimitrios.schoinianakis@nokia-bell-labs.com (Dimitrios Schoinianakis)

linear-domain values. Then, depending on the selected strategy (`lms-dda` or `lms-ddr`), the client either accumulates the linear intermediate results in plaintext or it re-encrypts them and sends them to the server for homomorphic accumulation in the linear domain respectively. This avoids CKKS bootstrapping entirely—the most expensive operation in fully homomorphic evaluation—by exploiting interaction to reset the computation domain at substantially lower cost [Gen09, CHK⁺18]. Even with recent advances that bring CKKS bootstrapping amortized cost below one millisecond [AKP25], bootstrapping still constrains parameter selection and circuit layout; the LNS approach sidesteps these constraints altogether.

The rest of the paper is organized as follows. Section 2 reviews CKKS and Section 3 introduces the logarithmic number system. In Section 4 the proposed LNS-CKKS protocols are presented and a security analysis is provided in Section 5. Section 6 reports the performance results and Section 7 discusses limitations of the proposed methodology. Conclusions are offered in Section 8.

2 CKKS Preliminaries

The CKKS scheme [CKKS17] is recalled at the level of detail needed for the present construction. Let N be a power of two and let $R = \mathbb{Z}[X]/(X^N + 1)$ denote the N -th cyclotomic ring. For a modulus q , write $R_q = R/qR$.

Encoding and encryption. CKKS encodes a vector $\mathbf{z} \in \mathbb{C}^{N/2}$ into a ring element via the (scaled) canonical embedding. A scaling factor $\Delta > 0$ is introduced to control precision: the encoder computes $\mathbf{pt} = \lfloor \Delta \cdot \sigma^{-1}(\mathbf{z}) \rfloor \in R$, where σ^{-1} denotes the inverse of the canonical embedding restricted to the $N/2$ conjugate-paired roots. Encryption produces a pair of ring elements $\mathbf{ct} = (\mathbf{ct}_0, \mathbf{ct}_1) \in R_q^2$ such that $\mathbf{ct}_0 + \mathbf{ct}_1 \cdot s \approx \mathbf{pt} \pmod{q}$ for a secret key $s \in R$.

Homomorphic operations. Ciphertext addition is a coordinate-wise ring addition: given $\mathbf{ct}, \mathbf{ct}'$ encoding $\mathbf{z}, \mathbf{z}'$ respectively,

$$\mathbf{ct}_{\text{add}} = \mathbf{ct} + \mathbf{ct}' \implies \text{Dec}(\mathbf{ct}_{\text{add}}) \approx \mathbf{z} + \mathbf{z}'.$$

This costs $\mathcal{O}(N)$ coefficient additions. Ciphertext multiplication is considerably more expensive. The tensor product $\mathbf{ct} \otimes \mathbf{ct}'$ produces a degree-2 ciphertext in R_q^3 that must be relinearized back to degree 1 via a key-switching operation `Relin`:

$$\mathbf{ct}_{\text{mul}} = \text{Relin}(\mathbf{ct} \otimes \mathbf{ct}') \implies \text{Dec}(\mathbf{ct}_{\text{mul}}) \approx \mathbf{z} \cdot \mathbf{z}'.$$

Key switching involves decomposing a ring element with respect to a gadget vector and multiplying by a (large) evaluation key, typically costing $\mathcal{O}(N \log q)$ operations. After multiplication, the scaling factor doubles from Δ to Δ^2 , so a rescaling step `Rescale` divides by Δ and drops the least-significant modulus from the chain:

$$\mathbf{ct}' = \text{Rescale}(\mathbf{ct}_{\text{mul}}) = \lfloor \Delta^{-1} \cdot \mathbf{ct}_{\text{mul}} \rfloor \pmod{q/q_0}.$$

Modulus chain and depth budget. CKKS operates with a chain of moduli $Q_L = \prod_{i=0}^L q_i$. Each rescaling drops one prime q_i , so a fresh ciphertext at level L supports at most L multiplications before the modulus is exhausted. Supporting depth d requires $L \geq d$ primes and a ring degree N large enough to achieve the target security level with modulus Q_L . In practice, deeper circuits require exponentially larger N , which directly increases ciphertext size and all operation costs. For example, as will be shown in subsequent sections, under the `HEStd_128_classic` setting in `OpenFHE` and for a given

target precision, a depth-8 multiplication chain requires approximately $N = 32,768$, while a depth-21 attention pipeline (simulating a typical LLM attention mechanism) requires $N = 65,536$ respectively.

Bootstrapping. To mitigate the increasing number of levels consumed by multiplications, CKKS bootstrapping [CHK⁺18] has been introduced, which homomorphically evaluates an approximation of the decryption circuit to refresh a ciphertext’s level without client interaction. While it enables unbounded depth in principle, bootstrapping itself consumes many levels and is extremely expensive (often dominating the entire computation). The framework presented here avoids bootstrapping entirely by using the interactive setting to cross arithmetic domains at a much lower cost.

3 LNS Preliminaries

The logarithmic number system (LNS) represents a nonzero real number x as a pair (s, ℓ) where $s \in \{-1, +1\}$ is the sign and $\ell = \log_b |x|$ is the logarithm of the magnitude with respect to a fixed base $b > 1$ [Par20, PS17]. The key algebraic property is that multiplication reduces to addition:

$$x \cdot y = (s_x s_y) \cdot b^{\ell_x + \ell_y}. \quad (1)$$

This is attractive for hardware and software implementations of multiply-intensive computations because addition is typically cheaper than multiplication.

Conversely, addition in LNS is non-trivial. For $x, y > 0$ with $\ell_x \geq \ell_y$, one can write

$$x + y = b^{\ell_x} (1 + b^{\ell_y - \ell_x}) = b^{\ell_x + \phi(\ell_y - \ell_x)},$$

where $\phi(z) = \log_b(1 + b^z)$ is the *Gaussian logarithm* [Par20, PS96, PS01]. Since ϕ is nonlinear, it cannot be evaluated by a native ring operation; in the HE context, it would require a polynomial approximation consuming multiplicative levels, which defeats the purpose of using LNS to avoid multiplications.

Definition 1 (Clamped LNS payload). For $x \in [-1, 1]$, base $b > 1$, and clamp threshold $\delta > 0$, define

$$\text{Log}_\delta(x) = \log_b(\max(|x|, \delta)), \quad \text{Sign}(x) = \text{sgn}(x).$$

The corresponding *LNS ciphertext payload* is the pair $\text{ct}_{\text{lns}} = (\text{ct}_\ell, s)$, where $\text{ct}_\ell = \text{Enc}(\text{Log}_\delta(x))$ is an encryption of the log-magnitude and $s = \text{Sign}(x)$ is plaintext sign metadata.

The clamp parameter δ prevents numerical instability when $|x|$ is very small (or zero). In the implementation considered here, $b = 2$ and $\delta = 10^{-12}$.

4 LNS-CKKS Protocols

4.1 General Framework

Consider a client-server model where the client holds private data and the server evaluates an encrypted computation. The server alternates between two phases. In the *LNS phase*, multiplication-heavy segments are evaluated by adding encrypted log-magnitudes on the server, consuming no multiplicative depth. In the *linear CKKS phase*, the protocol transitions back to standard CKKS when partial products must be accumulated (e.g., in a dot product or matrix multiplication), since additions are native in that domain.

Unless stated otherwise, confidentiality is considered against a semi-honest server in the standard outsourced-computation setting, so privacy of the client's data is inherited from CKKS. Correctness of the refresh step, however, assumes that the client returns honestly re-encrypted values; robustness against malicious client responses is not claimed and is revisited in Section 5.

The transition from LNS to linear CKKS is realized by a *refresh* operation:

Definition 2 (Refresh). Given an encrypted log-magnitude ct_ℓ and sign metadata s , the client performs

$$\text{Refresh}(\text{ct}_\ell, s) : \quad \tilde{z} \leftarrow s \cdot b^{\text{Dec}(\text{ct}_\ell)}, \quad \text{ct}_z \leftarrow \text{Enc}(\tilde{z}),$$

yielding a fresh ciphertext ct_z in the standard linear domain.

The refresh is the core interactive step. It requires the client to decrypt the log-domain ciphertext, exponentiate, apply the sign, and re-encrypt. At no point does the server learn the plaintext; it observes only encrypted payloads in both directions.

4.2 LNS Encoding and Multiplication

To encrypt a value x in LNS form, the client computes:

$$s_x = \text{Sign}(x), \tag{2}$$

$$\ell_x = \text{Log}_\delta(x) = \log_b(\max(|x|, \delta)), \tag{3}$$

$$\text{ct}_{\ell_x} = \text{Enc}(\ell_x). \tag{4}$$

The client sends $(\text{ct}_{\ell_x}, s_x)$ to the server.

To multiply two LNS-encoded values on the server:

$$\text{ct}_{\ell_{xy}} \leftarrow \text{ct}_{\ell_x} + \text{ct}_{\ell_y}, \quad s_{xy} \leftarrow s_x \cdot s_y. \tag{5}$$

This is a single ciphertext addition, i.e., zero multiplicative levels are consumed, plus a plaintext sign multiplication. Because no relinearization or rescaling is involved, the server requires no evaluation keys and performs only $\mathcal{O}(N)$ coefficient additions per multiplication, making the LNS phase particularly attractive for resource-constrained server environments.

4.3 Decrypt-Decode-Accumulate (DDA)

When the computation reaches a point requiring summation (e.g., an inner product $\sum_i x_i y_i$), the server has computed termwise LNS products $\{(\text{ct}_{\ell_i}, s_i)\}_{i=1}^n$. In the DDA strategy, the server sends all encrypted LNS terms to the client, which accumulates them locally. Concretely, the server transmits $\{(\text{ct}_{\ell_i}, s_i)\}_{i=1}^n$; the client decodes each term by computing $\tilde{z}_i = s_i \cdot b^{\text{Dec}(\text{ct}_{\ell_i})}$, accumulates $z = \sum_{i=1}^n \tilde{z}_i$ in plaintext, and—if a subsequent encrypted phase requires the result—re-encrypts $\text{ct}_z = \text{Enc}(z)$ in LNS or linear form.

DDA requires only a single server-to-client (S2C) transfer per accumulation boundary. The result is initially plaintext on the client, which is both an advantage (no further decryption needed if this is an output) and a limitation (re-encryption is needed if the server requires the accumulated value).

4.4 Decrypt-Decode-Re-encrypt (DDR)

In the DDR strategy, the client decodes the LNS terms as in DDA but re-encrypts them *individually* in linear CKKS form and sends them back to the server, which performs the accumulation in the encrypted domain. The server first transmits $\{(\text{ct}_{\ell_i}, s_i)\}_{i=1}^n$ to the

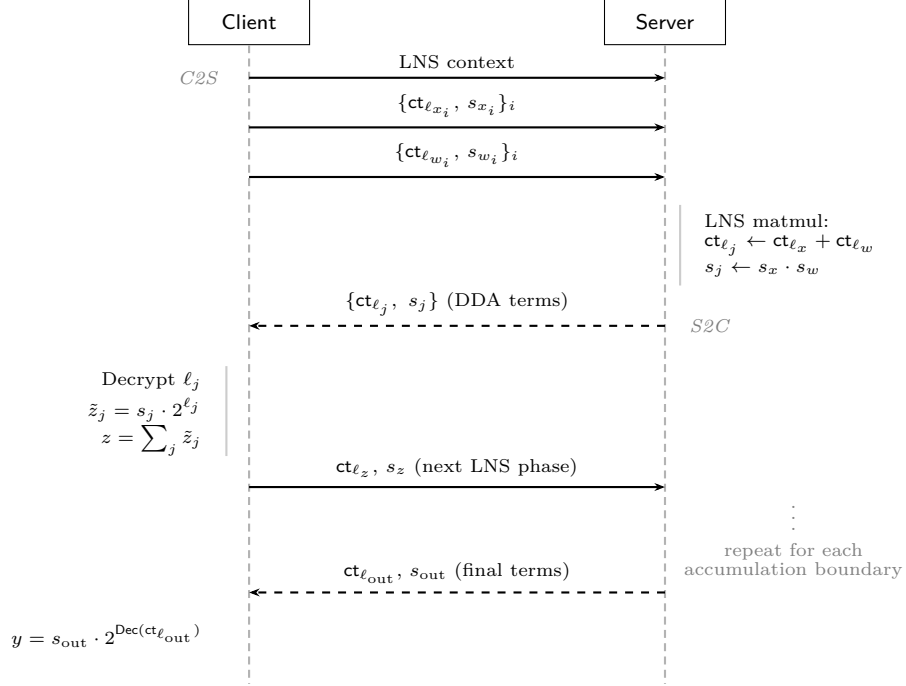


Figure 1: Simplified sequence diagram for the **lns-dda** protocol. The server computes termwise LNS products via ciphertext addition, then sends all terms to the client. The client decrypts, exponentiates, accumulates in plaintext, and re-encrypts for the next phase.

client, which decodes each term as $\tilde{z}_i = s_i \cdot b^{\text{Dec}(\text{ct}_{\ell_i})}$ and returns $\{\text{Enc}_{\text{linear}}(\tilde{z}_i)\}_{i=1}^n$. The server then accumulates homomorphically: $\text{ct}_z = \sum_{i=1}^n \text{Enc}_{\text{linear}}(\tilde{z}_i)$.

DDR therefore requires a round-trip (S2C + C2S) per accumulation boundary, plus server-side encrypted summation. The advantage is that the accumulated result remains as a ciphertext on the server, available for further encrypted computation without an additional upload.

When a subsequent pipeline stage requires re-entering the LNS domain, a *decode–encode–re-encrypt* (DER) transition is performed: the server sends the accumulated linear ciphertext ct_z to the client, which decrypts it, computes the LNS representation $(\ell_z, s_z) = (\text{Log}_\delta(z), \text{Sign}(z))$, and returns a fresh encrypted LNS payload $(\text{ct}_{\ell_z}, s_z)$. Each DER transition thus adds one additional round-trip and is a cost unique to DDR, since DDA produces the linear-domain result in plaintext on the client and can re-encode into LNS locally without communication.

4.5 Automatic Strategy Selection

The **lns-auto** strategy uses a cost model to select between DDA and DDR at each accumulation boundary. The planner estimates the end-to-end cost of each path by considering three factors: the communication cost (S2C payload alone for DDA vs. the combined S2C + C2S payload for DDR), the server-side computation (zero for DDA, since accumulation is performed on the client, vs. encrypted summation for DDR), and the downstream penalty (whether the next operation requires the result on the server, favoring DDR, or on the client, favoring DDA). In the benchmark suite considered here, **lns-auto** behaves similarly to **lns-dda** because the cost of the DDR round-trip and server-side summation rarely compensates for the downstream benefit.

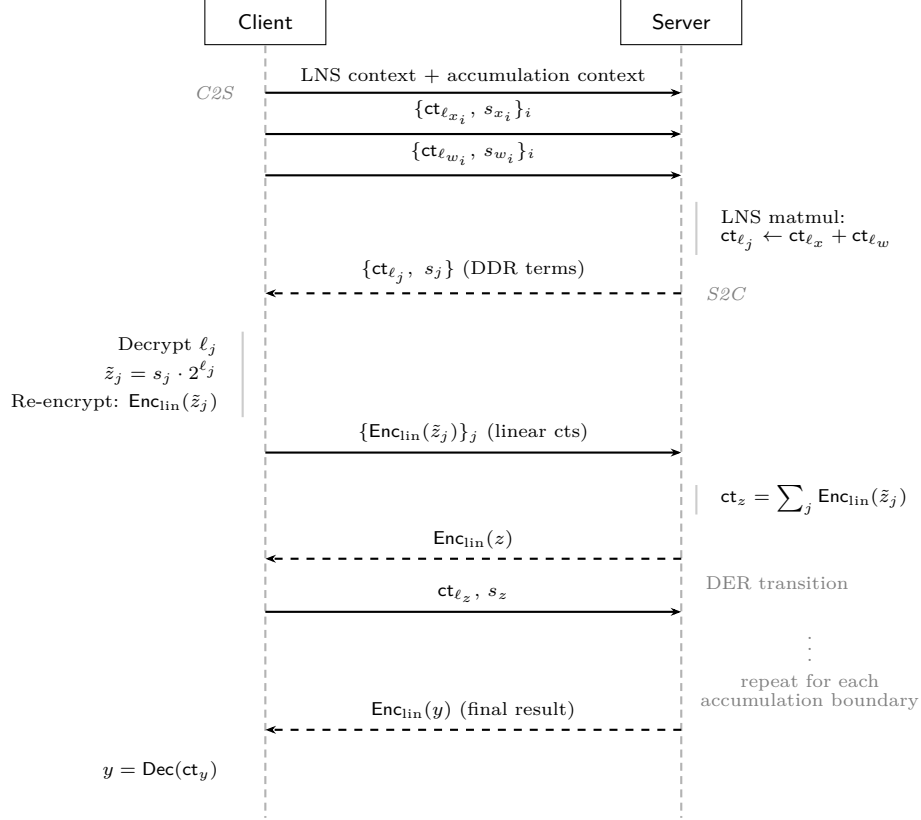


Figure 2: Simplified sequence diagram for the **lns-ddr** protocol. Unlike DDA, the client re-encrypts each decoded term individually in the linear CKKS domain and sends them back to the server. The server accumulates in the encrypted domain.

4.6 Comparison: DDA vs. DDR

Table 1 summarizes the structural differences between the two explicit strategies. The fundamental tradeoff is between communication efficiency and server-side state retention.

DDR is appropriate when the accumulated result must remain encrypted on the server for downstream operations (e.g., in a multi-stage pipeline where the server fuses outputs from different branches). However, DDR incurs several additional costs: roughly $2\times$ communication per accumulation boundary, due to the C2S upload of re-encrypted linear ciphertexts; server-side encrypted summation; a DER round-trip whenever the pipeline must re-enter LNS after accumulation; and a shallow linear accumulation context that adds up to the initial handshake. In the reported benchmarks, DDA consistently outperforms DDR in both latency and total payload, as quantified in Section 6.

5 Security Analysis

This section analyses the security of the protocols of Section 4. The protocols are presented as an abstract cryptographic functionality: they specify *what* the client and server compute and *which* values cross the network, but they delegate wire-level integrity and session authentication to the underlying transport (e.g. mutually authenticated TLS), as is

Table 1: Structural comparison of DDA and DDR strategies. DDA requires only one communication direction per accumulation boundary and a single LNS context, while DDR requires both directions plus an additional shallow linear accumulation context.

Property	DDA	DDR
Communication per boundary	S2C only	S2C + C2S
Accumulation location	Client (plaintext)	Server (encrypted)
Server computation	None at boundary	Encrypted sum
Result availability	Plaintext on client	Ciphertext on server
Payload scaling	$\sim n \cdot \text{ct}_{\text{ins}} $	$\sim n \cdot (\text{ct}_{\text{ins}} + \text{ct}_{\text{iin}})$
Context requirements	LNS only	LNS + shallow linear accumulation context
Authenticated-transport segments required per boundary	2 (one round-trip)	4 (two round-trips)

standard for homomorphic-encryption system designs.² The analysis below serves the purpose of establishing the core input-privacy guarantee formally, and stating precisely the assumptions under which the scheme attains that guarantee. In this way, a deployment can translate each assumption into a concrete engineering requirement.

The input-privacy guarantee is established with the aid of a symbolic verifier, **Verifpal** v0.51.0 [KNB20], a bounded Dolev–Yao-style protocol analyser. Three companion models are used: one for DDA, one for DDR, and one for DDA augmented with a signed transport.

Threat model. An active attacker may drop, reorder, replay, substitute, or forge arbitrary messages on every wire. CKKS ciphertexts are modelled as semantically secure public-key encryption under the client’s public key pk : without the secret key sk , the attacker cannot recover any plaintext from a ciphertext it observes. Server-side homomorphic operations (LNS log-sum, linear-domain accumulation) are modelled as deterministic combiners whose outputs the attacker can evaluate but cannot invert. The server is honest-but-curious: it executes the protocol but may inspect everything it receives. This is the standard adversary model for HE protocol analysis and matches the “outsourced computation on sensitive data” deployment target motivating most HE-related works.

Input privacy is preserved under the active adversary. The symbolic analysis confirms the informal security claim of Section 4: for both DDA and DDR, every secret input and every log-domain or linear-domain intermediate that crosses the network remains confidential against the active adversary. The client is the sole holder of sk and the protocol never exposes sk on the wire; the log-domain reductions and the server-side accumulations are functions of ciphertexts only, never of sk . Consequently, privacy of the client’s data is inherited from CKKS semantic security, and the LNS layer neither exposes new material nor interacts with the attacker’s decryption oracle. This is a stronger statement than “semi-honest server” and it holds even when the transport is untrusted, as long as the server itself does not deviate.

²By this we mean the standard layering of HE system designs (e.g. [JVC18, GBDL⁺16]): the cryptographic protocol is specified over an abstract reliable, integrity-protected, mutually authenticated bytestream, and a deployment instantiates that abstraction with mTLS or an equivalent.

Sign leakage. The confidentiality query against the logarithm signs failed, but this is due to the scheme’s intentional design: LNS sign bits are transmitted as plaintext metadata. Encoding signs inside a CKKS slot would demand either a bitwise-homomorphic scheme (unavailable in CKKS) or a polynomial-approximated indicator function consuming multiplicative levels, either of which would negate the depth savings that motivate LNS in the first place. The practical consequence is that one bit leaks per LNS-encoded slot per refresh. However, this channel does not carry the decision bit for the workloads reported in Section 6, so their confidentiality claim is unaffected; applications for which slot-level sign-of-output is sensitive (for example, a binary classifier whose verdict is precisely the sign of a scalar) should mitigate the channel in one of two ways: preprocess operands into a constant-absolute-value form that suppresses signs at the cost of dynamic range, or perform the final sign-revealing step inside a conventional CKKS context using standard bitwise constructions. In any case, the sign channel should be declared in any deployed system.

Deployment requirements on the transport. The protocols do not carry per-message authenticators in the specification; wire-level integrity is deferred to the transport. As such, two obligations follow, both satisfied by standard cryptographic mechanisms:

- (i) **Authenticated channel.** The client–server link must provide integrity and endpoint authentication, e.g. TLS with a pinned server identity, or mutually authenticated TLS when the server must also accept refresh uploads from the client (as in DDR).
- (ii) **Session binding on server replies.** When the server’s homomorphic reply is a deterministic function of the client’s request (which it is, since homomorphic operations are public), a replay of a prior session’s reply under its original signature is syntactically valid. To bind a reply to the current session, the server includes a fresh nonce in the signed payload, so that the authenticator covers both the request digest and the session nonce. This is a single extra field per reply; it has no effect on the computation and negligible effect on payload size.

DDR introduces one additional uplink (the re-encrypted refresh ciphertext) and one additional downlink (the final accumulation) per refresh boundary, and thus inherits the same obligations on these extra channels; this is recorded in Table 1.

Trust assumption on the refresh client. The protocol assumes an *honest refresh client*: intermediate LNS ciphertexts are decrypted, exponentiated, and correctly re-encrypted. A malicious client may return arbitrary ciphertexts and thereby violate correctness. Privacy of the client’s data is unaffected by client misbehaviour (the client is the privacy principal), but active robustness against a dishonest refresh client is not provided. Hardening this assumption is an orthogonal research direction: proof-carrying refresh (the client attaches a succinct argument that the re-encrypted value is consistent with the LNS intermediate), masked refresh (the server blinds the intermediate so the client cannot selectively corrupt specific slots), or a universally composable [Can01] ideal functionality abstracting the refresh as a trusted oracle are all natural next steps and are left to future work.

6 Performance Evaluation

6.1 Experimental Setup

All benchmarks were implemented in Python using the OpenFHE library (the security setting was set to `HEStd_128_classic`) [BAB⁺22]. Four variants are evaluated: *Linear* (standard leveled CKKS without LNS), *lns-auto* (LNS with automatic per-node DDA/DDR

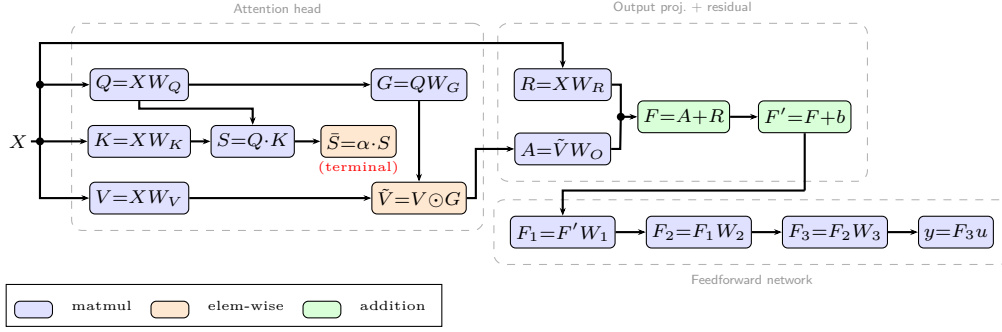


Figure 3: Dataflow graph of the attention-like pipeline benchmark. Nodes are color-coded by operation type: matrix multiplication (blue), element-wise multiplication (orange), and addition (green).

selection), **1ns-dda** (LNS with forced decrypt-decode-accumulate), and **1ns-ddr** (LNS with forced decrypt-decode-re-encrypt). Network latency is simulated under three models: *datacenter* (0.25 ms one-way latency, 25 Gbps bandwidth), *LAN* (1 ms, 1 Gbps), and *WAN* (20 ms, 0.1 Gbps).

Workloads. Two circuit families are studied at operand sizes $n \in \{8, 16, 32\}$. The first is a simple *matrix chain*: a sequence of L chained matrix multiplications (default $L=8$; $L \in \{8, 12, 16, 20\}$ is also swept in Section 6.4), designed to test whether LNS can eliminate deep multiplicative chains. The second is an *attention-like pipeline*: a structured circuit modeled after a single-head transformer attention block [VSP⁺17] followed by a feedforward network, chosen to represent a realistic AI inference workload.

Attention pipeline structure. The attention benchmark consists of 15 computation nodes operating on $n \times n$ matrices, organized into three stages. The *attention head* (7 nodes) computes query, key, and value projections $Q = XW_Q$, $K = XW_K$, $V = XW_V$ from input X , a dot-product score $S = Q \cdot K$, a scaled score $\bar{S} = \alpha \cdot S$, a gating matrix $G = QW_G$, and gated values $\tilde{V} = V \odot G$. The scaled score $\bar{S}$ is a terminal node: it contributes to the circuit’s multiplicative depth but is not consumed by any downstream operation, mirroring a branch that would feed a softmax in a full transformer but is pruned here for simplicity reasons. The *output projection and residual* stage (4 nodes) computes $A = \tilde{V}W_O$, a residual projection $R = XW_R$, their sum $F = A + R$, and a plaintext bias addition $F' = F + b$. The *feedforward network* (4 nodes) applies three successive linear layers $F_1 = F'W_1$, $F_2 = F_1W_2$, $F_3 = F_2W_3$, followed by a vector projection $y = F_3u$ with $u \in \mathbb{R}^{n \times 1}$. In total, the pipeline performs 9 matrix multiplications, 1 dot product, 2 element-wise multiplications, 2 additions, and 1 matrix–vector product. The dataflow graph is illustrated in Figure 3.

In the slot-packed representation used for bandwidth-efficient linear CKKS evaluation, a single matrix multiplication is not a single multiplicative level: **OpenFHE**’s **EvalMatMulSquare** routine packs the full $n \times n$ matrix into a single ciphertext using a diagonal encoding, then evaluates the product via a sequence of ciphertext–plaintext multiplications and rotations. Under **FLEXIBLEAUTO** rescaling strategy available in **OpenFHE**, each packed matmul consumes 2–3 levels (empirically: 2 for the first matmul from fresh ciphertexts, 3 for subsequent matmuls due to scale-alignment adjustments), while element-wise multiplications consume 1 level and additions consume 0. Tracing the critical path through the attention graph yields: $X \xrightarrow{+2} Q \xrightarrow{+3} G \xrightarrow{+1} \tilde{V} \xrightarrow{+3} A \xrightarrow{+0} F' \xrightarrow{+3} F_1 \xrightarrow{+3} F_2 \xrightarrow{+3} F_3 \xrightarrow{+3} y$, totaling exactly 21 levels. This multiplicative-level amplification is a

direct consequence of slot packing: without it, each matmul would require transmitting n^2 individual ciphertexts (one per scalar product), increasing payload substantially. Packing reduces communication to a single ciphertext per matrix but inflates the per-operation level cost, which in turn inflates the modulus chain and ring degree. This is precisely the mechanism by which LNS sidesteps the problem: in LNS mode, multiplications are additions on log-magnitudes and consume zero levels, so the context stays small regardless of circuit structure.

Relation to production transformers. The pipeline captures the core computational pattern of a single attention head plus feedforward block as found in GPT-2 (12 heads, $d_{\text{model}} = 768$, $d_{\text{ff}} = 3072$) and BERT ($d_{\text{model}} = 768$, $d_{\text{ff}} = 3072$) [VSP⁺17]. The $n \times n$ operands ($n \in \{8, 16, 32\}$) correspond to a scaled-down version of the per-head dimension ($d_k = 64$ in both architectures). Two deliberate simplifications are made for encrypted evaluation. First, the softmax normalization is replaced by a scalar scaling factor α and an element-wise gating mechanism, because softmax is a transcendental function whose polynomial approximation would consume many additional levels. In a standard transformer, the attention weights $\text{softmax}(QK^\top/\sqrt{d_k})$ are applied to V ; here, the score branch $S = Q \cdot K$, $\tilde{S} = \alpha \cdot S$ is retained in the graph to preserve the multiplicative-depth footprint of a real attention head, but $\tilde{S}$ is a terminal node whose output is not consumed—the role of attention weighting is instead absorbed by the gating branch $G = QW_G$, $\tilde{V} = V \odot G$. Second, the nonlinear activations (GELU in GPT-2, ReLU in BERT) between feedforward layers are omitted; the FFN layers are applied as three successive linear maps rather than the standard two-layer expand–contract pattern with an activation in between. These simplifications are standard in the HE literature: CryptoNets [GBDL⁺16] replaces activation functions with low-degree polynomials, while GAZELLE [JVC18] defers non-polynomial layers to interactive garbled-circuit evaluation between client and server; polynomial approximations for comparison and sign have also been studied [LLNK22]. The remaining arithmetic structure—projections, gating, residual skip, and multi-layer feedforward—faithfully reflects the dominant cost centers of transformer inference.

Context parameters. Table 2 summarizes the CKKS contexts used and the size of each public-context handshake. The key observation is the context asymmetry: the linear attention baseline requires $N = 65,536$ with a modulus chain of 23 primes, yielding about 2.9 GB of public context material, whereas all LNS variants run at $N = 8,192$ with 3–4 primes (about 50 MB).

Differences also exist with regards to the evaluation-key sets of the contexts. The linear context always requires a relinearization key, since every $\text{ct} \times \text{ct}$ operation (`mul`, `dot`, `matmul`) produces a degree-2 ciphertext that must be relinearized; under slot packing it additionally requires a plan-specific set of Galois (rotation) keys for the diagonal-encoded `EvalMatMulSquare` and for `EvalSumCols`/broadcast paths used during server-side accumulation. In contrast, the LNS context never requires a relinearization key, because no $\text{ct} \times \text{ct}$ operation is ever executed: LNS multiplication is a single ciphertext addition of log-magnitudes, and the termwise dot and matmul reductions are likewise sequences of additions. For the matrix-chain workload the LNS context therefore needs *no* evaluation keys at all. For the attention workload, whose plan includes a packed termwise matmul with encrypted transpose and column broadcast, the LNS context needs only a small set of Galois keys for those layout operations—still no relinearization key, since the matmul itself is scalar-sum reduction in the log domain.

Table 2: CKKS context parameters under `HEStd_128_classic` and the corresponding public-context handshake sizes (MB).

Pipeline	Linear CKKS context			LNS CKKS context		
	Parameters		Pub. ctx	Parameters		Pub. ctx
Matrix chain ($L=8$)	$N=32,768$, $\Delta=2^{40}$	$[60, 40^{\times 9}, 60]$	29.4 MB	$N=8,192$, $\Delta=2^{25}$	$[35, 25, 35]$	0.40 MB
Attention ($L=21$)	$N=65,536$, $\Delta=2^{40}$	$[60, 40^{\times 21}, 60]$	2,936 MB	$N=8,192$, $\Delta=2^{29}$	$[39, 29, 29, 39]$	50.5 MB

Table 3: Matrix-chain results ($L = 8$) under `HEStd_128_classic`. n : operand size. $\text{Sp}\times$: speedup vs. linear. All LNS variants achieve a $25.80\times$ payload reduction (identical contexts and message structure).

n	Variant	Datacenter			LAN			WAN		
		E2E (ms)	MAE	$\text{Sp}\times$	E2E (ms)	MAE	$\text{Sp}\times$	E2E (ms)	MAE	$\text{Sp}\times$
8	Linear	831	3.9e-9	1.0	1199	3.1e-9	1.0	4894	3.5e-9	1.0
	<code>lns-auto</code>	86	1.9e-7	9.7	116	2.6e-8	10.3	487	4.5e-8	10.1
	<code>lns-dda</code>	83	1.4e-7	10.0	112	1.0e-7	10.7	485	1.2e-7	10.1
	<code>lns-ddr</code>	83	1.9e-7	10.0	112	1.8e-7	10.7	485	9.0e-8	10.1
16	Linear	871	3.8e-9	1.0	1199	3.8e-9	1.0	4900	4.0e-9	1.0
	<code>lns-auto</code>	89	1.2e-7	9.7	115	2.4e-7	10.5	493	2.2e-8	10.0
	<code>lns-dda</code>	87	2.3e-8	10.0	113	2.0e-7	10.6	489	9.0e-8	10.0
	<code>lns-ddr</code>	86	6.1e-8	10.2	112	3.0e-8	10.7	487	7.5e-8	10.1
32	Linear	885	3.0e-9	1.0	1241	3.4e-9	1.0	4960	4.6e-9	1.0
	<code>lns-auto</code>	94	1.2e-7	9.4	115	1.9e-7	10.8	492	2.2e-7	10.1
	<code>lns-dda</code>	89	1.4e-7	10.0	119	1.0e-7	10.4	490	1.3e-7	10.1
	<code>lns-ddr</code>	88	8.7e-8	10.1	119	2.3e-7	10.5	490	7.3e-8	10.1

6.2 Matrix Chain Results

Table 3 reports the matrix-chain measurements. All three LNS variants achieve consistent speedups of $9.4\times$ – $10.8\times$ across all network settings and problem sizes, with a uniform payload reduction of $25.80\times$ and MAE below 2.4×10^{-7} . The three strategies perform nearly identically because the matrix chain has only a single accumulation boundary, making the DDA/DDR distinction negligible.

6.3 Attention Pipeline Results

Table 4 shows a substantially larger separation on the attention workload. The linear baseline ranges from 98 s ($n=8$, datacenter) to 477 s ($n=32$, WAN), driven by the large ring degree and deep modulus chain. DDA/auto achieve speedups of 34 – $36\times$ at $n=8$, declining to 22 – $32\times$ at $n=32$; DDR is consistently 1.5 – $2\times$ slower.

The DDA–DDR gap is most visible in payload (Figure 4): DDA achieves $41.2\times$ reduction at $n=8$, growing to $46.5\times$ at $n=32$ as the linear-to-LNS context ratio widens, whereas DDR is limited to 22 – $26\times$ due to the round-trip communication. LNS approximation error grows with size: at $n=32$, MAE reaches $\sim 4.5 \times 10^{-3}$ for DDA and $\sim 7.5 \times 10^{-3}$ for DDR, compared with 4.5×10^{-4} for the linear baseline.

Table 4: Attention pipeline results under `HESD_128_classic`. Payload reduction grows with size: DDA/auto achieve $40.5\times$ at $n=8$, $43.2\times$ at $n=16$, and $46.2\times$ at $n=32$; DDR achieves $22.4\times$, $24.3\times$, and $26.4\times$ respectively. DDR is consistently $1.5\text{--}2\times$ slower than DDA.

n	Variant	Datacenter			LAN			WAN		
		E2E (ms)	MAE	Sp $\times$	E2E (ms)	MAE	Sp $\times$	E2E (ms)	MAE	Sp $\times$
8	Linear	98137	3.7e-5	1.0	106736	3.6e-5	1.0	130855	3.8e-5	1.0
	lms-auto	2868	3.5e-4	34.2	3070	3.8e-4	34.8	5665	3.0e-4	23.1
	lms-dda	2695	3.9e-4	36.4	3106	3.3e-4	34.4	5566	3.1e-4	23.5
	lms-ddr	4895	3.1e-4	20.1	5585	7.2e-4	19.1	10969	5.2e-4	11.9
16	Linear	218263	9.4e-5	1.0	221184	9.1e-5	1.0	244128	9.3e-5	1.0
	lms-auto	6194	1.1e-3	35.2	6711	1.0e-3	33.0	10834	9.4e-4	22.5
	lms-dda	6258	9.2e-4	34.9	6674	7.4e-4	33.1	10632	1.2e-3	23.0
	lms-ddr	11666	1.9e-3	18.7	12274	3.0e-3	18.0	21182	2.0e-3	11.5
32	Linear	457144	4.5e-4	1.0	450610	4.6e-4	1.0	477499	4.6e-4	1.0
	lms-auto	14399	4.8e-3	31.7	15101	4.2e-3	29.8	21829	4.5e-3	21.9
	lms-dda	14116	4.3e-3	32.4	15305	4.4e-3	29.4	21454	4.2e-3	22.3
	lms-ddr	26235	7.2e-3	17.4	28118	5.9e-3	16.0	43537	7.5e-3	11.0

Table 5: Depth scaling on the matrix chain ($n=8$). ; PL $\times$ is Linear/DDA.

L	Ring degree N		DC latency (ms)		WAN latency (ms)		Total payload (MB)			
	Linear	LNS	Linear	DDA	Linear	DDA	Linear	DDA	Sp $\times_{DC}$	PL $\times$
8	32 768	8 192	875	91	4 903	494	78.15	3.03	9.6	25.8
12	32 768	8 192	1 570	129	9 510	685	134.80	4.08	12.2	33.0
16	65 536	8 192	5 343	170	31 167	890	420.57	5.14	31.4	81.9
20	65 536	8 192	7 865	224	46 642	1 114	605.15	6.19	35.1	97.8

6.4 Depth Scaling

To measure how the LNS advantage scales with circuit depth, the matrix-chain length is swept over $L \in \{8, 12, 16, 20\}$ at fixed $n=8$. The LNS context ($N=8,192$, [35, 25, 35]) is held constant across all runs while the linear context is scaled to accommodate each depth.

Table 5 reports the results; the payload column includes the public context handshake plus every encrypted and plaintext transport message observed during the run. At $L=8$ and $L=12$ the linear context fits at $N = 32,768$ and speedup grows modestly from $9.6\times$ to $12.2\times$. At $L=16$, where the modulus budget exceeds the `HESD_128_classic` capacity for $N = 32,768$, a jump to $N = 65,536$ is required. This ring-degree doubling causes a discontinuous increase in linear latency (1.6s to 5.3s on datacenter) while DDA grows smoothly (129ms to 170ms), widening the speedup from $12\times$ to $31\times$ and the payload reduction from $33\times$ to $82\times$. At $L=20$ the gap reaches $35\times$ speedup and $98\times$ payload reduction, while the LNS context remains at $N = 8,192$ with 3 primes throughout.

6.5 Packed vs. Unpacked Linear Evaluation

To confirm that the depth-21 requirement originates from slot packing, the attention pipeline ($n=8$) is re-evaluated with packing disabled: each matrix element is encrypted individually, so every matrix multiplication reduces to scalar products consuming one level each. The linear context shrinks from $N=65,536$ at depth 21 to $N=32,768$ at depth 9.

Despite the halved ring degree, disabling packing *increases* datacenter latency from

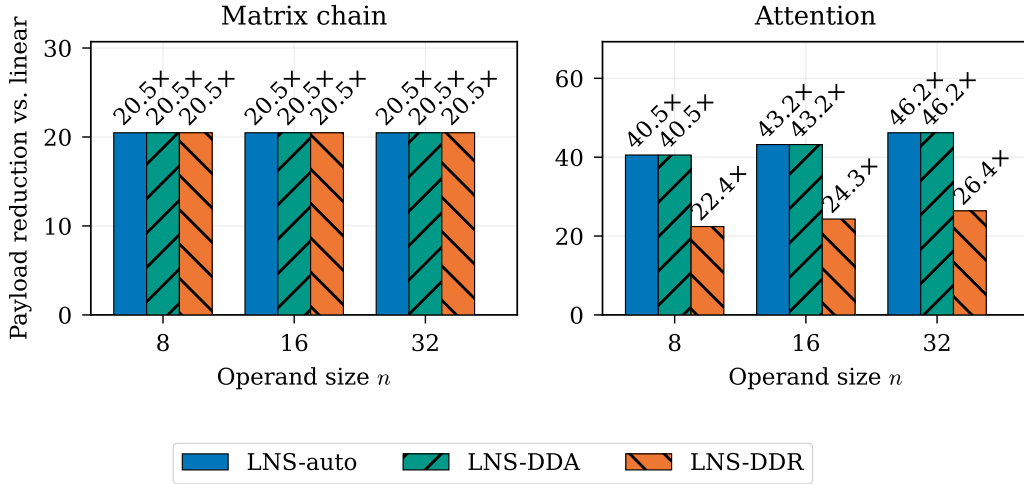


Figure 4: Payload reduction factor vs. linear CKKS baseline across operand sizes $n \in \{8, 16, 32\}$ for both pipelines. DDA/auto and DDR achieve identical reduction on the matrix chain ($25.8\times$) but diverge on the attention pipeline, where DDA reaches 41–46 $\times$ while DDR is limited to 22–26 $\times$.

Table 6: Attention pipeline ($n=8$, datacenter): packed vs. unpacked linear CKKS and LNS DDA. Disabling slot packing reduces the depth from 21 to 9 and halves the ring degree, but increases latency by $2.9\times$ and payload by $1.2\times$. LNS is unaffected by the packing mode and achieves $36\times$ speedup vs. packed or $106\times$ vs. unpacked linear.

Mode	N	Depth	DC (ms)	Payload (GB)	MAE
Linear (packed)	65 536	21	98 137	3.24	$3.7e-5$
Linear (unpacked)	32 768	9	285 749	3.78	$1.8e-6$
LNS DDA	8 192	2	2 695	0.08	$3.9e-4$

98 s to 286 s ($2.9\times$ slower) because per-element evaluation forgoes the SIMD parallelism of diagonal-encoded packed matmul. Payload also grows slightly (3.78 GB vs. 3.24 GB) as n^2 individual ciphertexts at $N=32,768$ collectively exceed a single packed ciphertext at $N=65,536$. Accuracy improves by an order of magnitude (MAE 1.8×10^{-6} vs. 3.7×10^{-5}) because the unpacked path avoids diagonal-encoding approximation error. LNS is unaffected by the packing choice: DDA speedup grows from $36\times$ (vs. packed) to $95\times$ (vs. unpacked), confirming that the LNS advantage is robust across linear-baseline implementation strategies.

7 Limitations

Live-client requirement. The refresh of Definition 2 is performed by the client, so every LNS-to-linear domain crossing requires the client to be online and reachable while the server is computing. The scheme therefore does not target fully non-interactive deployments in which the server must complete the computation autonomously (e.g. long-running batch jobs scheduled when the client is offline). This is the cost of avoiding bootstrapping and whether it is acceptable depends on the deployment use-case and system requirements.

Domain crossing overhead. Addition in the log domain requires the Gaussian logarithm ϕ (Section 3), which is transcendental and would cost multiplicative levels to

approximate homomorphically. The implemented system therefore performs every accumulation in the linear domain via a refresh round-trip. The end-to-end advantage of LNS is consequently a function of the workload’s multiplications-to-additions ratio: circuits dominated by additions, or those that interleave the two densely, see diminishing returns because each accumulation pays one network round-trip.

Error accumulation. The LNS approximation error grows with problem size, as seen in the $n=32$ attention results ($\text{MAE} \sim 10^{-3}$ to 10^{-2}). This reflects both the clamping error of Definition 1 and the accumulation of CKKS rounding noise across multiple refresh cycles. Tighter clamping thresholds and higher-precision LNS contexts (larger Δ) mitigate this at the cost of slightly increased context size.

8 Conclusions and Future Work

An `OpenFHE`-based implementation of an LNS acceleration framework for CKKS has been presented, in which multiplication-heavy subcircuits are evaluated by encrypted additions over log-magnitudes and a client-assisted refresh maps the computation back into linear CKKS when accumulation is required. The central practical insight is that this design *decouples context requirements from circuit depth*: the LNS context stays at $N=8,192$ irrespective of multiplication-chain length, whereas linear CKKS must escalate N as depth grows. The resulting context asymmetry yields substantial speedups and payload reductions as was demonstrated on matrix-chain and attention-like workloads.

Three directions for future work are envisioned. First, efficient polynomial approximations of the Gaussian logarithm or a homomorphically-friendly version of LUT-based methods for LNS addition would extend the range of circuits that can remain in LNS form without crossing into the linear domain for every addition. Second, evaluation should be extended to full transformer blocks and hybrid plans that lower only selected multiplicative regions into LNS, supported by an autotuned `lns-auto` cost model calibrated against measured network and ciphertext-layout costs. Finally, the symbolic security analysis of Section 5 can be lifted to an unbounded-session proof in `ProVerif` [Bla16] or the `Tamarin Prover` [MSCB13], and the refresh oracle hardened against a malicious client through proof-carrying or masked variants formalized in a Universally Composable (UC) framework [Can01].

References

- [AKP25] Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. General functional bootstrapping using CKKS. In *Advances in Cryptology – CRYPTO 2025*, 2025. Cryptology ePrint Archive, Report 2024/1623.
- [BAB⁺22] Ahmad Al Badawi, Andreea Alexandru, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Carlo Pascoe, Yuriy Polyakov, Ian Quah, Saraswathy R. V., Kurt Rohloff, Jonathan Saylor, Dmitriy Suponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. OpenFHE: Open-source fully homomorphic encryption library. Cryptology ePrint Archive, Report 2022/915, 2022.
- [Bla16] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, 2016.

- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
- [CHK⁺18] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 360–384. Springer, Cham, April / May 2018.
- [CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 409–437. Springer, Cham, December 2017.
- [GBDL⁺16] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 201–210, 2016.
- [Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st ACM STOC*, pages 169–178. ACM Press, May / June 2009.
- [JVC18] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. GAZELLE: A low latency framework for secure neural network inference. In *27th USENIX Security Symposium*, pages 1651–1669, 2018.
- [KNB20] Nadim Kobeissi, Georgio Nicolas, and Karthikeyan Bhargavan. Verifpal: Cryptographic protocol analysis for the real world. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 76–95, 2020.
- [LLNK22] Eunsang Lee, Joon-Woo Lee, Jong-Seon No, and Young-Sik Kim. Minimax approximation of sign function by composite polynomial for homomorphic comparison. In *IEEE Transactions on Dependable and Secure Computing*, volume 19:6, pages 3711–3727, 2022.
- [MSCB13] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The TAMARIN prover for the symbolic analysis of security protocols. In *Computer Aided Verification (CAV)*, pages 696–701, 2013.
- [Par20] Behrooz Parhami. Computing with logarithmic number system arithmetic: Implementation methods and performance benefits. *Computers & Electrical Engineering*, 87:106800, 2020.
- [PS96] Vassilios Paliouras and Thanos Stouraitis. A novel algorithm for accurate logarithmic number system subtraction. In *IEEE International Symposium on Circuits and Systems (ISCAS)*, volume 4, pages 268–271, 1996.
- [PS01] Vassilios Paliouras and Thanos Stouraitis. Low-power properties of the logarithmic number system. In *15th IEEE Symposium on Computer Arithmetic (ARITH-15)*, pages 229–236, 2001.
- [PS17] Vassilios Paliouras and Thanos Stouraitis. Logarithmic number system. In *Arithmetic Circuits for DSP Applications*. Wiley, 2017.

- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 5998–6008, 2017.